DATABASES - INFORMATION SYSTEMS III

OVERVIEW ON RELATIONAL DATABASES

LIST OF CONTENTS.

LECTURER & CO-ORDINATOR

Wilhelm C. Rothman

DATABASES - INFORMATION SYSTEMS III

1. C O U R S E O V E R V I E W.

The course will provide an introduction to relational databases. The theory of databases as well as practical applications will be covered.

2. A I M S O F T H E C O U R S E.

2.1. To provide a theoretical foundation about database management systems and the relational database approach in particular.
2.2. To provide the student with hands-on experience in ORACLE SQL.
2.3. To provide practical examples in order to expand on the theoretical.

3. S Y L L A B U S.

LECTURE 1 INTRODUCTION TO THE RELATIONAL MODEL.

(05-02)

Aim: To explain the concept of a table and the relations among tables.
1. Architecture of a relational database.
2. The table and terminology (e.g. attribute, tuple) and relationships

Upon completion, the student should be able to:
* define all related terms.
* Know the cardinality of relationships and how to represent it diagramatically

LECTURE 2 SYSTEMS ANALYSIS & DESIGN

*** Basic Logical Data Structure diagrams & Connectivity**

LECTURE 3 RELATIONAL ALGEBRA

*** Know the composition and query structures of the Relational Algebra (Select, Project & Join, etc.)**

LECTURE 4 & 5 NORMALIZATION.

Aim: The student must understand and apply the definitions of the normalization process.

1. Definitions on FD & FFD & TD.
2. Zero'th form & 1 NF & Anomalies.
3. 2 NF & Anomalies.
4. Keys: Determinant, Candidate, Primary, Foreign.
5. 3 NF & BCNF & Anomalies.
6. Project on Normalization. (SET RELATION)

Upon completion, the student should be able to:
* define the terms FD, FFD, TD, 1NF etc.
* transform an example relation from 1NF to BCNF.
* Evaluate normal forms of relations.

LECTURE 6 & 7 SQL

Structured Query Language (SQL)

USAGE AND SYNTAX OF SQL

Structured Query Language

Aims:

Upon completion the student should be able to:

* Be familiar with SQL
* Solve (problems) queries with SQL
* Rewrite queries using SQL

R E S O U R C E S.

Prescribed : **Database Systems Management and Design** - Third Edition.

Philip J. Pratt & Adamski
(Boyd & Fraser Publishing Company, 1994)
ISBN : 0-87709-115-3

Additional notes : will be handed out in class.

References:

C. J. Date: **An Introduction to Database Systems** 3rd Edition.

Litton: **Database Management Systems** A Guide to SQL. (Philp J. Pratt, South-Western Publishers, Boston, 1991)

Wilhelm C. Rothman

DATABASES - INFORMATION SYSTEMS III

LECTURE 1

1.1 HISTORY OF DATABASE MANAGEMENT

APOLLO project 1960 (J.F. Kennedy)

IBM developed Generalized Update Acces Method

GUAM developed into DL/1 in 1966 (IMS)

Mid-1960s - IDS (Integrated Data Store) by General Electric Charles Bachman headed team. CODASYL (COBOL developers) developed a standard for DBMSs and formed a DBTG to do just that.

Dr E.F. Codd developed **RELATIONAL DATABASES** in 1970, which became commercial in 1980s

In 1970 to 1980 many enhancements were added:

Data Dictionaries, Report Generators, Query facilities, Non-Procedural Languages.

1.2 STANDARD SET BY ANSI/SPARC THREE SCHEMA ARCHITECTURE

1.2.1 EXTERNAL LEVEL

1.2.2 CONCEPTUAL or LOGICAL LEVEL

1.2.3 INTERNAL or PHYSICAL LEVEL

1.3 DATAMODEL

1.3.1 TWO COMPONENTS: STRUCTURE & OPERATIONS

STRUCTURE = Way system structures the data. That is the way the users **PERCEIVE** the data.

OPERATIONS = Facilities within the DBMS to manipulate the database

1.3.2 CATEGORIES OF DATA MODELS

1.3.2.1 HIGH-LEVEL or CONCEPTUAL

1.3.2.2 LOW-LEVEL or PHYSICAL

1.3.2.3 IMPLEMENTATION DATA MODEL

1.4 RELATIONSHIPS, ENTITIES & ATTRIBUTES

1.4.1 ENTITY & ATTRIBUTE

Attribute is a property of an entity.

1.4.2 RELATIONSHIPS

1:1	ONE TO ONE
1:M	ONE TO MANY
M:N	MANY TO MANY

If more than one value found in a relation of another relation, then one to many.

Example: FACULTY MEMBER and DEPARTMENT

DEPARTMENT <----->> FACULTY MEMBER

1.4.3CODD's CONTRIBUTIONS

1.4.3.1 CODD's SYSTEM CLASSIFICATIONS

1.4.3.2 CODD's RELATIONAL RULES

1.4.3.3 SET-THEORY TO MANIPULATE TABLES

Draw diagram for FACULTY database.

CATEGORIES OF SYSTEMS

1. TABULAR

The only structure perceived by the user is the table, but the system does not support SELECT, PROJECT, and JOIN operators in the unrestricted fashion indicated in Codd's defintion. Inverted file systems typically fall in this category.

2. MINIMALLY RELATIONAL

The system supports the tabular structure, together with SELECT, PROJECT and JOIN, but not the complete set of operations from the relational algebra.

3. RELATIONALLY COMPLETE

Any system that supports the tabular structure and all the operations of the relational algebra.

4. FULLY RELATIONAL

A system as in 3, as well as supporting the two integrity rules (entity and referential integrity).

CODD's TWELVE RULES

0. Relational Capabilities

A relational DBMS must be able to manage data entirely through its relational capabilities, not having to use non-relational capabilities.

1. Information

All data should be presented by values in tables. This not only includes the data in the database, but also the information about the database (Catalog).

2. Access

Users should be able to access any value in the database by giving the name of the table, the name of the column, and a value for the primary key of the table. In non-relational systems, users are often forced to use linked lists or sequential scanning of a database for a certain value.

3. Nulls

A relational DBMS should support null data values.

4. Catalog

A relational DBMS should furnish a user-accessible catalog. Users must be able to access this catalog using the same relational features they use to access data in a database.

5. Data Sublanguage

A relational DBMS must furnish at least one language capable of supporting all the folllowing: data definition, view definition, data manipulation, integrity constraints, authorizations, and logical transactions. (Many non-relational DBMSs have one language for defining a database, another for defining views, still another for manipulating data, and so on.)

6. Updatable Views

Any view that is updatable in theory must be updatable by a relational DBMS.

7. High-Level Update

A relational DBMS should furnish facilities for retrieving, adding, updating, or deleting a set of rows with a single command. (In most nonrelational DBMSs, users are limited to accessing a single row at a time)

8. Physical Data Independence

In a relational DBMS, changes to the physical storage and/or access methods used in a database should not impact users or application programs. The creation or dropping of an index, for example, would not change the way users interact with the database. Nor would it necessitate changes in application programs.

9. Logical Data Independence

In a relational DBMS, changes to the logical structure of the database should not impact users or application programs. This is provided, of course, that the user's view of data is still legitimate with the new structure. If a change invalidates a particular user's view, clearly there is no way the DBMS can prevent the user from being affected. For example, the removal from the database of a column required by the user will clearly have an impact on the user.

10. Integrity

The data sublanguage must support the definition of integrity constraints. These constraints must be stored in the system catalog and enforced by the DBMS, not by application programs.

11. Distribution

A distributed database is a database that is stored on computers at several sites of a computer network, and in which users can access data at any site in the network. A relational DBMS taht manipulates a distributed database should allow users and programs to access data at a remote site in exactly the same fashion they do at the local site.

1.5 **RELATION or TABLE**

A relation is a two dimensional table with the following properties:

1. The entries in the table are single valued.
2. Each column has a distinct name(attribute)
3. All values within a column belong to same attribute name.
4. The order of columns is immaterial.
5. Each row is distinct.
6. The order of rows is immaterial.

1.6 **TERMINOLOGY**

SYSTEMS ANALYSIS & DESIGN

RELATIONAL DATABASE

NETWORK DATABASE & PROGRAMMING

EXERCISE 1

1.1 Define Redundancy.
1.2 Define entity, attribute & relationship.
1.3 How does the ANSI/X3/SPARC model relate to data independence?
1.4 What contribution did Dr. E.F. Codd make to the Database area?
1.5 What is a datamodel? List 4 main data models.

LECTURE 2

LOGICAL DATA STRUCTURE TECHNIQUE (LDS)

SYNONYMS:

- Entity Model
- Logical Model
- Data Model
- Data Structure **Diagram**
- Softbox Diagram

The LDS technique enables one to classify data and relationships between data.

BUILDING BLOCKS OF AN LDS

The LDS technique is concerned with the identification and classification of entities, their attributes and the relationships between them.

ENTITIES

An **entity** is something (tangible or non-tangible) that is of such importance in the application area, that the organization wishes to keep information about it. Your company employs **EMPLOYEE**s in different **DEPARTMENT**s. Distinguish between an entity type and entity occurrences. DEPARTMENTT and EMPLOYEE are entity types and Peter, Mary, Accounts Department are entity occurrences.

ATTRIBUTES

Attributes are data elements describing an entity. Attributes can be split into IDENTIFYING and DATA attributes.

The entities and their attributes represent a logical view of the data, whereas records and fields represent its physical implementation.

RELATIONSHIPS

Entity occurrences in an environment are related to each other:

department EMPLOYS employee

DEPARTMENT

EMPLOYEE

Relationships can be classified intto:

ONE TO ONE	(1:1)
ONE TO MANY	(1:m)
MANY TO MANY	(m:n)

Relationships represent **VERBS** in any description of processes in the environment:

wholesalers **SUPPLIES** product
parent **HAS** children

Many to Many relationships must be resolved into ONE tto MANY relationships.

DEPARTMENT

EMPLOYEE

DOCUMENTING THE ENTITY MODEL

1. Each entity is represented by an oval box, with the entity name in the top portion

2. Relationships connecting entities may be 1:1, 1:n, or m:n.

3. Many to many relationships will only be recorded in interim versions of the entity model. All m:n relationships must be converted to 1:m relationships.

4. In m:n relationships the relationship is said to connect one MASTER entity occurrences to many DETAIL entity occurrences.

5. Each relationship is named, usually a verb. The sentence should read from top to bottom or left to right. If it is read in opposite direction use brackets.

STEPS IN THE LDS TECHNIQUE

1. Select an initial set of entities from the environment.
2. Investigate and record inter-relationships, using the LDS grid.
3. Convert the LDS grid into an interim LDS.
4. Convert each n:m relationship into two 1:n relationships.
5. Validate the structure shown in the model against system requirements.
6. Rationalise the structure.
7. Revalidate the structure (if anything changed after step 6)

Please note that steps 5 to 7 are omitted when an LDS of the current system is constructed.)

EXAMPLE

PROCESSING REQUIREMENTS

1. Find all the students taking a specific subject.

2. Find all the students taking a specific course.
3. Find all the students taught by a specific lecturer.
4. Find the courses where a specific subject is taken.
5. Give a list of all courses, together with its compulsory subjects.
6. Find all the courses presented by a particular school.
7. Find all the students enrolled in a particular school.

ENTITIES identified:

STUDENT, COURSE, SUBJECT, LECTURER, SCHOOL

Construct a matrix, with the names in the same order on the horisontal and vertical.

	STUDENT	COURSE	SUBJECT	LECTURER	SCHOOL
STUDENT					
COURSE	X				
SUBJECT	X	X			
LECTURER			X		
SCHOOL		X			

Only Direct relationships must be indicated. If a relationship may be expressed in terms of another entity type, that entity can act as a link.

Student and School is not directly related, as well as Lecturer and Course.

An interim data model for Students Records System:

STUDENT LETURER

COURSE SUBJECT

SCHOOL

CONVERT ALL M:N RELATIONSHIPS

STUDENT YEAR-MARK LETURER

MANDATORY

OFFERING

SUBJECT

COURSE SUBJECT

SCHOOL

ENTITY RELATIONSHIP MODEL

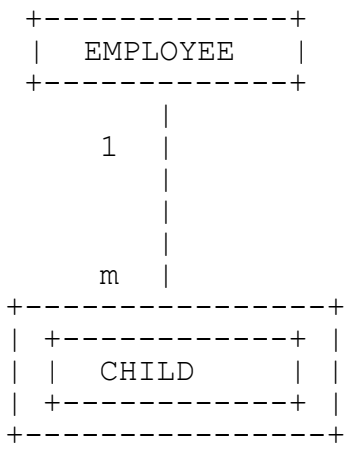
Developed by Peter Chen. It gives a graphical approach to Database Design. Representation of entities and relationships. Both may carry attributes.

TYPES OF DEPENDENCIES

1. Existence
2. ID-Dependency

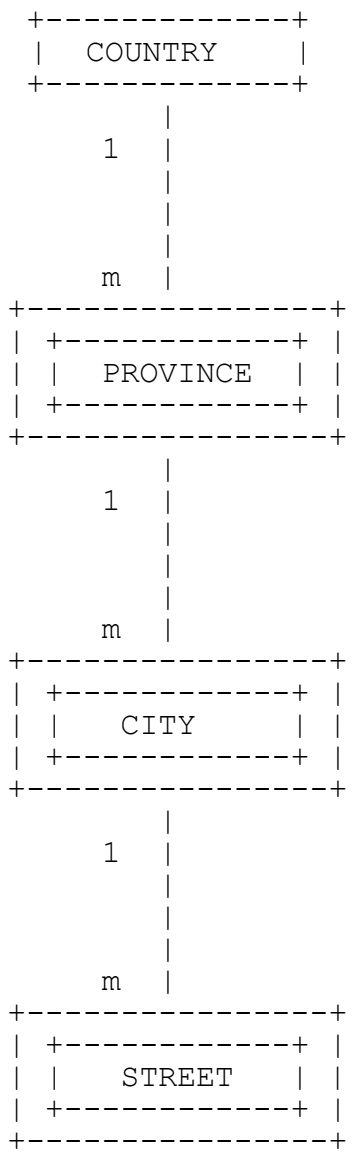
1. EXISTENCE DEPENDENCY

The existence of an entity occurrence depends on the existence of another entity



2. ID-DEPENDENCY

The entity cannot uniquely be defined by its own attributes and it has to be identified by the relationships with other entities.

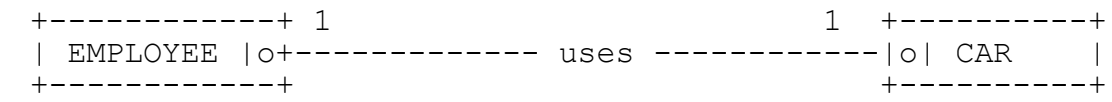


Any relationship expressed by the membership class and the degree of the relationship.

MEMBERSHIP: **Obligatory, non-obligatory(Optional)**

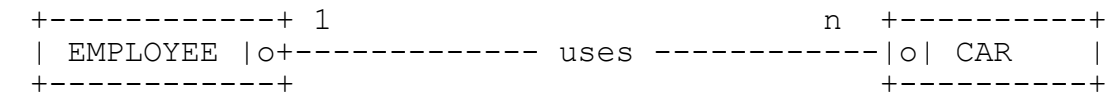
DEGREE: **1:1, 1:m, m:n**

KEYS: **PRIMARY, ALTERNATE, CANDIDATE, FOREIGN, SECONDARY**



EMPLOYEE (EMP#, EMPNAME, CAR#, MODEL,...)

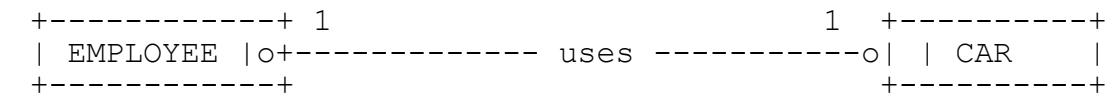
AK



EMPLOYEE (EMP#, EMPNAME, ...)

CAR (CAR#, MODEL, ..., **EMP#**)

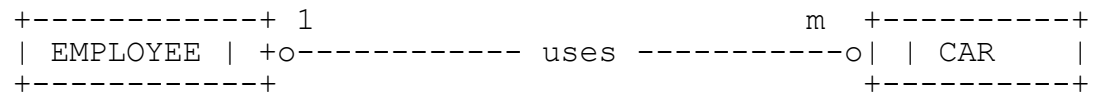
FK



EMPLOYEE (EMP#, EMPNAME, ...)

CAR (CAR#, MODEL, ..., **EMP#**)

AK

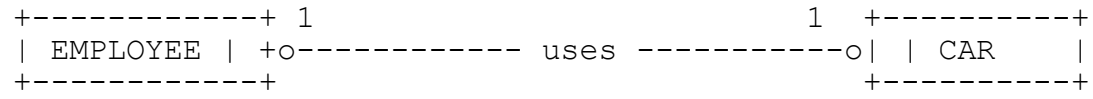


EMPLOYEE (EMP#, EMPNAME, ...)

CAR (CAR#, MODEL, ...)

CAR_EMP (CAR#, EMP#)

PK FK



EMPLOYEE (EMP#, EMPNAME, ...)

CAR (CAR#, MODEL, ...)

CAR_EMP (CAR#, EMP#)

PK AK

COMPOSITE AND COMPOUND KEYS

COMPOSITE - "Consisting of different parts of material"

Many to Many relationship

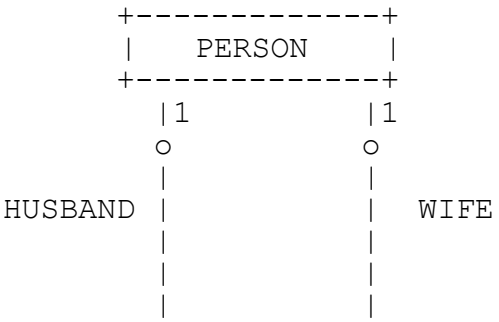
COMPOUND - "Consisting of two or more elements chemically united"

Weak Entity Types

RECURSIVE RELATIONSHIPS

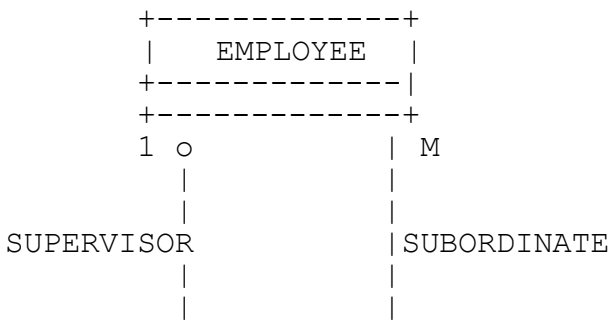
1. ONE TO ONE

Person can be married or not



PERSON (PERSON#, ...)
MARRIAGE (WIFE#, HUSBAND#, ...)

2. ONE TO MANY



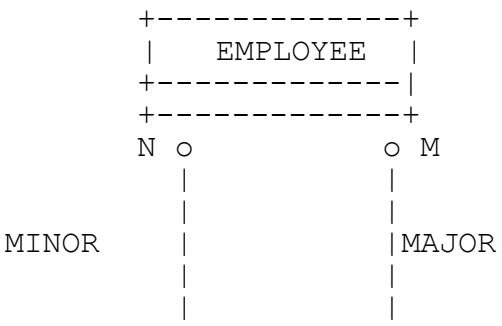
An employee may play a role of supervisor and/or the role of subordinate. Some employees do not supervise others and most

senior is regarded as self supervised.

EMPLOYEE (EMP#, ..., SUPER#)

3. MANY TO MANY

PART AND SUB-PART



STRUCTURE

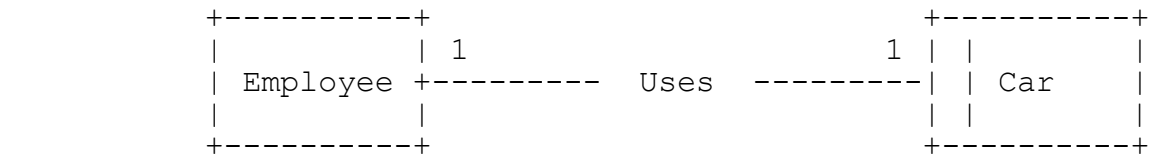
PART (PART#, ...)

STRUCTURE (MAJOR#, MINOR#, ...)

EXERCISE 2

2.1 **(5 MARKS)**

STUDY THE FOLLOWING SKELETON E-R DIAGRAM AND RELATIONS.



Employee (Emp#, Empname, ... other attributes.., ...)

Car (Car#, make, model, ... other attributes.., Emp#)

Why is Emp# posted to Car, instead of Car# posted to Employee? Motivate your answer.

2.2 (5 MARKS)

Company employees may optionally be members of the company's sports club. An employee is udentified by an Emp# and a sports club member is separately identified by a Membership#.

Draw up a skeleton E-R model showing the relationship between entity types Employee and Member.

Show how the relationship between the entities is represented by stating the relations with their important attributes.

2.3 **(10 MARKS)**

PUBLISHING DATABASE

A publishing company produces a number of specialist journals and papers. The company employs general editors who each take sole responsibility for editing one or more publications, each of which addresses a small number of subjects. Each author reports to one editor, but may submit a copy for the publications of other editors. Authors are specialists in certain subjects, and it is company policy to try to keep more than one specialist for each subject.

2.3.1Write down all the entities involved.

2.3.2Use the **Entity Relationship Model** and draw a skeleton conceptual diagram to depict the PUBLISHING DATABASE.

LECTURE 3

3. RELATIONAL ALGEBRA

Relational Algebra and Relational Calculus are theoretical ways of manipulating a Relational Database. Relational Algebra is relationally complete, i.e. anything that can be accomplish by relational calculus can also be done in Relational Algebra.

3.1 BASIC BUILDING BLOCKS

SELECT - HORIZONTAL
PROJECT - VERTICAL (NO DUPS)
JOIN - JOINING OF TWO TABLES

Each output of any command is directed to a file using:
GIVING FILENAME

SELECT: Horizontal subset of the relation

Example 1

List all information from the PACKAGE relation concerning package SS11

Example 2

List all information from the PACKAGE relation concerning those packages whose type is Database.

Example 3

List the ID, name, and version of all packages.

Example 4

List the ID, name, and version of all packages whose type is Database.

JOIN

Natural Join: Tables are joined on a common column, but one of the columns is ignored in the final result.

Equijoin: If both copies of the common column are maintained in final result.

Theta-Join: If tables are joined on a condition other than equality.

Outer Join: If the non matching rows are also included, but the columns of the joining table are left blank.

Example 5:

List the tag number and computer ID of all PCs together with the number and name of the employee to whom the PC is assigned.

UNION - Union compatibility is a prerequisite

Example 6:

List the computer ID and manufacturer name of all computers which either have a 486DX processor or have been assigned for home use, or both.

INTERSECTION

Example 7:

List the computer ID and manufacturer name of all computers that have a 486DX processor and have been assigned for home use.

DIFFERENCE

Example 8:

List the computer ID and the manufacturer name of all computers that have a 486DX processor but that have not been assigned for home use.

PRODUCT

Cartesian product as in maths.

DIVISION

SOFTWARE

PACKID	**TAGNUM**
AC01	32808
DB32	32808
DB32	37691
DB33	57772
WP08	32808
WP08	37691
WP08	57772
WP09	59836
WP09	77740

PACKAGE

PACKID
AC01
DB32

COMPUTER

COMPID	MFGNAME	MFGMODEL	PROCTYPE
B121	BANTAM	48X	486DX
B221	BANTAM	48D	486DX2
C007	CODY	D1	486DX
M759	LEMMIN	GRL	486SX

EMPLOYEE

EMPNUM	EMPNAME	EMPPHONE
124	ALVAREZ, RAMON	1212
567	FEINSTEIN, BETTY	8716
611	DINH, MELISSA	2963

PC

TAGNUM	COMPID	EMPNUM	LOCATION
32808	M759	611	ACCOUNTING
37691	B121	124	SALES
57772	C007	567	INFO SYSTEMS
59836	B221	124	HOME
77740	M759	567	HOME

PACKAGE

PACKID	PACKNAME	PACKVER	PACKTYPE	PACKCOST
DB32	MANTA	1.50	DATABASE	380.00
DB33	MANTA	2.10	DATABASE	430.18
SS11	LIMITLESS VIEW	5.30	SPREADSHEET	217.95
WP08	WORDS & MORE	2.00	WORD PROC	185.00
WP09	FREEWARE PROCESS	4.27	WORD PROC	30.00

SOFTWARE

PACKID	TAGNUM	INSTDATE	SOFTCOST
AC01	32808	09/13/95	754.95
DB32	32808	12/03/95	380.00
DB32	37691	06/15/95	380.00
DB33	57772	05/27/95	412.77
WP08	32808	01/12/96	185.00
WP08	37691	06/15/95	227.50
WP08	57772	05/27/95	170.24
WP09	59836	10/30/95	35.00
WP09	77740	05/27/95	35.00

EXERCISE 3

Given Relations:

PERSON (**IDNO**, NAME, AGE, INCOME, MARITAL_STATUS)
CAR (**CARID**, MANUFACTURER, MODEL, YEAR, COLOUR)
PERSON_HOME (**IDNO**, **HOMETYPE**)
HOME (**HOMETYPE**, NO_OF_ROOMS, NO_OF_BATHS, SWIM_POOL, PLOT_SIZE)
PERSON_CAR (**IDNO**, **CARID**)

Use **Relational Algebra** to perform the following queries on the given relations:

3.1 List the CARID and MODEL of all red cars manufactured by "M1" during 1980.

3.2 List the HOUSETYPE of all homes with PLOT_SIZE = 1000 m^2, owned by people older than 40, and in posession of a car manufactured by "M2".

3.3 List the HOUSETYPE and NAME of the owner(UNMARRIED), who owns a 4-bedroomed house with no swimming-pool.

LECTURE 4 & 5 NORMALIZATION

1. REDUNDANCY vs DUPLICATION

2. AIM OF NORMALIZATION

3. ANOMALIES CAUSED BY:
   - Delete
   - Update
   - Insert

4. STEPS IN NORMALIZATION

   Consider relation in 0NF

   Evaluate any anomalies

4.1 NORMALIZE INTO 1NF

   1NF
   1. ATOMIC FORM
   2. FUNCTIONAL DEPENDENCY

   EVALUATE FOR ANY ANOMALIES

   4.2 NORMALIZE INTO 2NF

   2NF
   1. MUST BE IN 1NF
   2. FULL FUNCTIONAL DEPENDENCY
   3. NO PARTIAL DEPENDENCY

   EVALUATE FOR ANY ANOMALIES

   4.3 NORMALIZE INTO 3NF

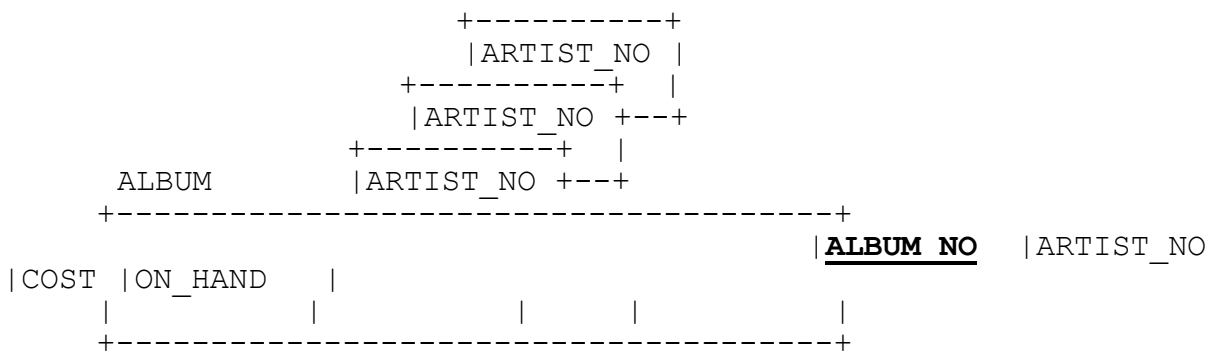
   3NF
   1. MUST NE IN 2NF
   2. NO TRANSITIVE DEPENDENCY
   3. REMOVE ANY CALCULATED FIELDS (ATTRIBUTES)

   EVALUATE FOR ANY ANOMALIES

   4.4 NORMALIZE INTO BCNF
   EVERY DETERMINANT MUST BE A KEY.

EXERCISE 4

4.1 The following conceptual file called ALBUM, shows the ARTIST_NO as a repeating field.



4.1.1 Explain the meaning of a repeating group.

4.1.2 Construct a table out of the ALBUM file and populate it with four rows of fictitious data to illustrate your answer at 4.1.1.

4.1.3 Normalize the relation to the highest normal form. Motivate your answer.

4.2

EMPLOYEE-SUPERVISOR TABLE
(EMPNO+SKILL = PRIMARY KEY)

EMPNO	**POSITON**	**SKILL**	**YRS**	**PROJ**	**SUPER**	**AGE**
AKIN	DESIGNER	PROLOG	2	RESERVE	ODELL	26
AKIN	DESIGNER	DBASE IV	4	SALES ANALYSIS	WIRTZ	26
KING	ANALYST	COBOL	4	PAYROLL	KEHOE	34
KING	ANALYST	PASCAL	3	PAYROLL	KEHOE	34
TURBAN	PROGRAMMER	COBOL	9	INVENT. CONTROL	WIRTZ	46
TURBAN	PROGRAMMER	RPG	11	RESERVE	ODELL	46
WIRTZ	PROJECT MANAGER	ADA	3	SALES ANALYSIS	WIRTZ	41
WIRTZ	PROJECT MANAGER	UNIX	4	SALES ANALYSIS	WIRTZ	41

EMPNO + SKILL - EMPNO AND SKILL (KEY OF TABLE)
YRS - YEARS
PROJ - PROJECT
SUPER - SUPERVISOR

Study the EMPLOYEE-SUPERVISOR table above and answer the following questions:

4.2.1 Give the current normal form of the EMPLOYEE-SUPERVISOR relation.

4.2.2 Using the definitions and rules of normalization motivate your answer at 4.2.1.

4.2.3 Discuss the anomalies which basic file operations may have on the EMPLOYEE-SUPERVISOR relation.

4.2.4 Normalize the relation to the highest normal form. Support your answer with definitions.

LECTURE 6 & 7 SQL - STRUCTURES QUERY LANGUAGE

DML = Data Manipulation Language = Necessary for querying the database.

Relational Calculus = Most important approach in manipulating databases.

A language is relationally complete if any relation that can be retrieved by Relational Calculus can also be retrieved by that language.

SQL = transform-oriented language or transform inputs into desired outputs.

SQL = can be used for querying as well as a Data Definition Language (DDL).

List the number, name and balance of all customers.

```
SELECT CUSTNUMB, CUSTNAME, BALANCE
     FROM CUSTOMER
```

List the complete part table.

```
SELECT *
          FROM PART
```

What is the name of customer 124?

```
SELECT CUSTNAME
     FROM CUSTOMER
     WHERE CUSTNUMB = 124
```

Find the customer for any customer whose name is Sally Adams.

```
SELECT CUSTNUMB
          FROM CUSTOMER
          WHERE CUSTNAME = 'ADAMS, SALLY'
```

COMPOUND CONDITIONS

List the descriptions of all parts in warehouse 3, and have more than 100 units on hand.

```
SELECT PARTDESC
     FROM PART
     WHERE WRHSNUMB = 3
     AND UNONHAND > 100
```

List the descriptions of all parts that are not in warehouse 3.

```
SELECT PARTDESC FROM PART
     WHERE NOT (WRHSNUMB = 3)
```

Find the available credit for all customers who have a credit limit of at least $800.

```
SELECT CUSTNUMB, CUSTNAME, (CREDLIM - BALANCE)
     FROM CUSTOMER
     WHERE CREDLIM >= 800
```

SORTING

Sorting can be done by using the ORDER BY clause.

List the name, number and address of all customers. Order the output by name.

```
SELECT CUSTNUMB, CUSTNAME, CUSTADDR
     FROM CUSTOMER
     ORDER BY CUSTNAME
```

How many parts are in item class HW?

```
SELECT COUNT(*)
     FROM PART
     WHERE ITEMCLSS = 'HW'
```

Find the number of customers and the total of their balances.

```
SELECT COUNT(*), SUM(BALANCE)
     FROM CUSTOMER
```

NESTING QUERIES

The inner query will be evaluated first.

List the customer number and name of all customers of Premiere Products who have a credit limit that is equal to the largest credit limit awarded to any customer of salesrep 3

```
SELECT MAX(CREDLIM)
     FROM CUSTOMER
     WHERE SLSRNUMB = 3


SELECT CUSTNUMB, CUSTNAMNE
     FROM CUSTOMER
     WHERE CREDLIM = 1000
```

NESTED QUERY

```
SELECT CUSTNUMB, CUSTNAME
     FROM CUSTOMER
     WHERE CREDLIM IN
          (SELECT MAX(CREDLIM)
               FROM CUSTOMER
                    WHERE SLSRNUMB = 3)
```

Because the output is only one value (1000) one could have use = instead of IN

GROUPING

List the total for each order.

```
SELECT ORDNUMB, SUM(NUMORD * QUOTPRCE)
     FROM ORDLNE
     GROUP BY ORDNUMB
     ORDER BY ORDNUMB
```

List the order total for those orders amounting to more than $200.

```
SELECT ORDNUMB, SUM(NUMORD * QUOTPRCE)
     FROM ORDLNE
     GROUP BY ORDNUMB
     HAVING SUM(NUMORD * QUOTPRCE) > 200
     ORDER BY ORDNUMB
```

JOINING OF TABLES

Determine the linking or foreign/primary keys.

List the number and name of each customer together with the number and name of sales rep who represents the customer.

```
SELECT CUSTNUMB, CUSTNAME, SLSREP.SLSRNUMB, SLSRNAME
     FROM CUSTOMER, SLSREP
     WHERE CUSTOMER.SLSRNUMB = SLSREP.SLSRNUMB
```

List the number and name of each customer whose credit limit is $800, together with the number and name of sales rep who represents the customer.

```
SELECT CUSTNUMB, CUSTNAME, SLSREP.SLSRNUMB, SLSRNAME
     FROM CUSTOMER, SLSREP
     WHERE CUSTOMER.SLSRNUMB = SLSREP.SLSRNUMB
     AND CREDLIM = 800
```

UNION OF TABLES

Union compatible means that columns of the two table must be of the same type.

List the number and name of customers who are either represented by sales rep 12, or who currently have orders on file, or both

```
SELECT CUSTNUMB, CUSTNAME
     FROM CUSTOMER
     WHERE SLSRNUMB = 12

UNION

SELECT CUSTOMER.CUSTNUMB, CUSTNAME
     FROM CUSTOMER, ORDERS
          WHERE CUSTOMER.CUSTNUMB =
                ORDERS.CUSTNUMB
```

```
rem  Wilhelm Rothman May 1995: Added Premiere REM  Product
Database
set termout off
rem host write sys$output "Building demostration tables.
Please
wait"
host echo "Building demonstration tables.  Please wait"
set feedback off


DROP TABLE PART;
DROP TABLE ORDLINE;
DROP TABLE ORDERS;
DROP TABLE CUSTOMER;
DROP TABLE SLSREP;

DROP TABLE EMP;
DROP TABLE DEPT;
DROP TABLE BONUS;
DROP TABLE SALGRADE;
DROP TABLE DUMMY;

create table part
     (partnumb char(4),
     partdesc char(10),
     unonhand number(4,0),
     itemclss char(2),
     wrhsnumb number(2,0),
     unitprce number(6,2));

INSERT INTO part VALUES
     ('AX12','IRON',104,'HW',3,17.95);
INSERT INTO part VALUES
     ('AZ52','SKATES',20 ,'SG',2,24.95);
INSERT INTO part VALUES
     ('BA74','BASEBALL',40,'SG',1,4.95);
INSERT INTO part VALUES
     ('BH22','TOASTER',95,'HW',3,34.95);
INSERT INTO part VALUES
     ('BT04','STOVE',11,'AP',2,402.99);
INSERT INTO part VALUES
     ('BZ66','WASHER',52,'AP',3,311.95);
INSERT INTO part VALUES
     ('CA14','SKILLET',2,'HW',3,19.95);
INSERT INTO part VALUES
     ('CB03','BIKE',44,'SG',1,187.50);
INSERT INTO part VALUES
     ('CX11','MIXER',112,'HW',3,57.95);
INSERT INTO part VALUES
     ('CZ81','WEIGHTS',208,'SG',2,108.99);
```

```
create table ordline
     ( ordnumb number(5,0),
     partnumb char(4),
     numord number(3,0),
     quotprce number(6,2));
INSERT INTO ordline VALUES
     (12489,'AX12',11,14.95);
INSERT INTO ordline VALUES
     (12491,'BT04',1,402.99);
INSERT INTO ordline VALUES
     (12491,'BZ66',1,311.95);
INSERT INTO ordline VALUES
     (12494,'CB03',4,175.00);
INSERT INTO ordline VALUES
     (12495,'CX11',2,57.95);
INSERT INTO ordline VALUES
     (12498,'AZ52',2,22.95);
INSERT INTO ordline VALUES
     (12498,'BA74',4,4.95);
INSERT INTO ordline VALUES
     (12500,'BT04',1,402.99);
INSERT INTO ordline VALUES
     (12504,'CZ81',2,108.99);


create table orders
     (
     ordnumb number(5,0),
     orddte date,
     custnumb number(3,0));

INSERT INTO orders VALUES
     (12489,'02-SEP-91',124);
INSERT INTO orders VALUES
     (12491,'02-SEP-91',311);
INSERT INTO orders VALUES
     (12494,'04-SEP-91',315);
INSERT INTO orders VALUES
     (12495,'04-SEP-91',256);
INSERT INTO orders VALUES
     (12498,'05-SEP-91',522);
INSERT INTO orders VALUES
     (12500,'05-SEP-91',124);
INSERT INTO orders VALUES
     (12504,'05-SEP-91',522);


CREATE TABLE CUSTOMER
     (CUSTNUMB NUMBER(4) NOT NULL,
     CUSTNAME VARCHAR(15),
     custaddr char(25),
```

```
     balance number(7,2),
     credlim number(4,2),
     slsrnumb number(2,0)
     );

INSERT INTO customer VALUES
     (124,'ADAMS, SALLY','481 OAK, LANSING,
          MI',418.75,500,3);

INSERT INTO customer VALUES
     (256,'SAMUELS, ANN','215 PETE, GRANT,
     MI',10.75,800,6);
INSERT INTO customer VALUES
     (311,'CHARLES, DON','48 COLLEGE, IRA,
     MI',200.10,300,12);
INSERT INTO customer VALUES
     (315,'DANIELS, TOM','914 CHERRY, KENT,
     MI',320.75,300,6);


INSERT INTO customer VALUES
     (405,'WILLIAMS, AL','519 WATSON, GRANT,
     MI',201.75,800,12);
INSERT INTO customer VALUES
     (412,'ADAMS, SALLY','16 ELM, LANSING,
     MI',908.75,1000,3);
INSERT INTO customer VALUES
     (522,'NELSON, MARY','108 PINE, ADA,
     MI',49.50,800,12);
INSERT INTO customer VALUES
     (567,'BAKER, JOE','808 RIDGE, HARPER,
     MI',201.20,300,6);
INSERT INTO customer VALUES
     (587,'ROBERTS, JUDY','512 PINE, ADA,
     MI',57.75,500,6);
INSERT INTO customer VALUES
     (622,'MARTIN, DAN','419 CHIP, GRANT,
     MI',575.50,500,3);



CREATE TABLE SLSREP
     (SLSRNUMB NUMBER(2,0) NOT NULL,
     SLSRNAME VARCHAR(15),
     SLSaddr char(25),
     TOTCOM number(7,2),
     COMMRATE number(3,2),
     DEPTNO NUMBER(2)
     );
```

```
INSERT INTO SLSREP VALUES
     (3,'JONES, MARY','123 MAIN, GRANT,
     MI',2150.00,.05,30);

INSERT INTO SLSREP VALUES
     (6,'SMITH, WILLIAM','102 RAYMOND, ADA,
     MI',4912.50,.07,30);
INSERT INTO SLSREP VALUES
     (12,'BROWN, SAM','419 HARPER, LANSING,
     MI',2150.00,.05,30);

CREATE TABLE EMP
       (EMPNO NUMBER(4) NOT NULL,
     ENAME VARCHAR2(10),
     JOB VARCHAR2(9),
     MGR NUMBER(4),
     HIREDATE DATE,
     SAL NUMBER(7,2),
     COMM NUMBER(7,2),
     DEPTNO NUMBER(2));

CREATE TABLE DEPT
       (DEPTNO NUMBER(2),
     DNAME VARCHAR2(14) ,
     LOC VARCHAR2(13) ) ;


INSERT INTO EMP VALUES
(7369,'SMITH','CLERK',7902,'17-DEC-80',800,NULL,20);

INSERT INTO EMP VALUES
(7499,'ALLEN','SALESMAN',7698,'20-FEB-81',1600,300,30);


INSERT INTO EMP VALUES
(7521,'WARD','SALESMAN',7698,'22-FEB-81',1250,500,30);

INSERT INTO EMP VALUES
(7566,'JONES','MANAGER',7839,'2-APR-81',2975,NULL,20);

INSERT INTO EMP VALUES
(7654,'MARTIN','SALESMAN',7698,'28-SEP-81',1250,1400,30);

INSERT INTO EMP VALUES
(7698,'BLAKE','MANAGER',7839,'1-MAY-81',2850,NULL,30);

INSERT INTO EMP VALUES
(7782,'CLARK','MANAGER',7839,'9-JUN-81',2450,NULL,10);

INSERT INTO EMP VALUES
     (7788,'SCOTT','ANALYST',7566,'09-DEC-82',3000,NULL,20);
```

```
INSERT INTO EMP VALUES
(7839,'KING','PRESIDENT',NULL,'17-NOV-81',5000,NULL,10);

INSERT INTO EMP VALUES
(7844,'TURNER','SALESMAN',7698,'8-SEP-81',1500,0,30);

INSERT INTO EMP VALUES
(7876,'ADAMS','CLERK',7788,'12-JAN-83',1100,NULL,20);

INSERT INTO EMP VALUES
(7900,'JAMES','CLERK',7698,'3-DEC-81',950,NULL,30);

INSERT INTO EMP VALUES
(7902,'FORD','ANALYST',7566,'3-DEC-81',3000,NULL,20);

INSERT INTO EMP VALUES
(7934,'MILLER','CLERK',7782,'23-JAN-82',1300,NULL,10);


INSERT INTO DEPT VALUES
     (10,'ACCOUNTING','NEW YORK');
INSERT INTO DEPT VALUES  (20,'RESEARCH','DALLAS');
INSERT INTO DEPT VALUES
     (30,'SALES','CHICAGO');
INSERT INTO DEPT VALUES
     (40,'OPERATIONS','BOSTON');

CREATE TABLE BONUS
     (
     ENAME VARCHAR2(10)  ,
     JOB VARCHAR2(9)  ,
     SAL NUMBER,
     COMM NUMBER
     ) ;

CREATE TABLE SALGRADE
      ( GRADE NUMBER,
     LOSAL NUMBER,
     HISAL NUMBER );

INSERT INTO SALGRADE VALUES (1,700,1200);
INSERT INTO SALGRADE VALUES (2,1201,1400);
INSERT INTO SALGRADE VALUES (3,1401,2000);
INSERT INTO SALGRADE VALUES (4,2001,3000);
INSERT INTO SALGRADE VALUES (5,3001,9999);

CREATE TABLE DUMMY
      ( DUMMY NUMBER );

INSERT INTO DUMMY VALUES (0);
```

```
COMMIT;

EXIT;
```

JOINS - INNER OR OUTER

INNER JOIN

Simple JOIN based on matching column values of a row.

OUTER JOIN

```
SELECT CUSTNUMB, ORDNUMB
     FROM CUSTOMER, OUTER ORDERS
          WHERE CUSTOMER.CUSTNUMB = ORDERS.CUSTNUMB;
```

Rows from the dominant table are retrieved without considering the dominant table, but rows from the subservient table are only retrieved if they satisfy the join condition. NULL values will fill the screen if values are not present in the subservient(diensbare) table.

OUTER JOIN THE RESULT OF A SIMPLE JOIN TO A THIRD TABLE

```
SELECT custname, orders.ordnumb, ordline.partnumb
     FROM CUSTOMER, outer (ORDERS,ORDLINE)
          WHERE customer.custnumb = orders.custnumb
          AND orders.ordnumb = ordline.ordnumb;
```

USE ALIASs IF TABLENAMES ARE TOO LONG

```
SELECT CUSTNAME, ORDDTE
     FROM CUSTOMER A, ORDERS B
          WHERE A.CUSTNUMB = B.CUSTNUMB;
```

WRITE SQL STATEMENTS TO ANSWER THE FOLLOWING QUERIES

1. List the number and date of those orders that do not contain part bt04.

2. List the customer number, the order number, the order date, and the order total for all orders with a total of more than R100.00. The listing must be ordered according to order number.

3. List the descriptions of all parts included in order 12491.

4. List all the numbers and dates of those orders that include a part located in warehouse 3.

5. List the salesrep numbers for all salesreps representing more than 3 customers.

6. Change query 5 and display the salesrep name as well.

7. List the customers with the same Surname (Name) but different customer numbers.

8. Give the order numbers of the orders placed by customer 124 on 09/05/91.

9. List the order number(s) and date for all orders that contain a line for an IRON.

10. List all the orders that were either placed on 90291 by a customer with a R1000.00 credit limit.

11. Find the total number of units on hand in warehouse 3.

12. List the customer(s) who placed an order(s) for any appliances (AP).

VIEWS

Existing permanent tables = base tables. A view is a pseudotable. It appears to the user to be an actual table. The view is actually a defined query.

1. Define a VIEW HSEWRES1-n, that consists of the part number, description, units on hand, and unit price of all parts of item class HW.

```
CREATE VIEW HSEWRES1 AS
     SELECT partnumb, partdesc, unonhand, unitprce
          FROM part
          WHERE itemclss = 'HW';
```

PLEASE NOTE THAT THE TABLE OPTION SHOWS THE VIEW AS A TABLE.

QUERYING A VIEW

The query is first merged with query that defines the view.

```
SELECT * FROM HSEWRES1 WHERE UNONHAND > 100;
```

2. Define a view as in the previous example, but give each selected column a different name.

```
CREATE VIEW HSEWRES1 (PNUM, PDESC, ONHAND, PRICE)
     AS SELECT PARTNUMB, PARTDESC, UNONHAND, UNITPRCE
          FROM PART
               WHERE ITEMCLSS = 'HW';
```

3. Define a view, SLSCUST, that consists of the salesrep number (called SNUMB), sales rep name (called SNAME), customer number (called CNUMB), and customer name (called CNAME) for all sales reps and matching customers in SLSREP and CUSTOMER tables.

4. Define a view, CREDCST1, that consists of a credit limit (CREDLIM) and the number of customers who have this limit (NUMBCUST)

```
CREATE  view CREDCST1 (Credlim, numbcust)
     as select credlim, count(*)
          from customer
          group by credlim;
```

ADVANTAGES OF VIEWS

1. They provide data independence
2. Since each user has his or her own view, the same data can be viewed by different users in different ways.
3. Columns not needed by the user should not be included and thus impossible for the user to access sensitive data.

DISADVANTAGES OF VIEWS

1. JOINed tables can cause problems during update. Some columns in the underlying table are not visible or the origin is uncertain.

2. Statistical or Calculated fields can cause a problem: TOT = (QTY * QUOTPRCE) - How will the view distinguish which attribute must be assigned which value.

3. Primary keys may be omitted causing a table to refuse a row of values.

If one join two base tables having the same primary key, updates are possible

WRITE SQL STATEMENTS TO ANSWER THE FOLLOWING QUERIES

1. A view, SMLLCUST1-n, is to be defined. It consists of customer number, name, address, balance, and credit limit for all customers whose credit limit is R500 or less.

1.1 Write a view definition for SMLLCUST.
1.2 Write an SQL query to retrieve the number and name of all customers in SMLLCUST whose balance is over their credit limit.
1.3 Convert the query from 1.2 to the query that will actually be executed.

2. A view CUSTORD, is to be defined. It consists of the customer number, name, balance, order number, and order date for all orders currently on file.

2.1 Write the view definition for CUSTORD.
2.2 Write an SQL query to retrieve the customer number, name, order number, and order date for all orders in CUSTORD for customers whose balance is more than R100.

3. Define a view ORDTOT, which consists of the order number and order total for each order currently on file in which the total is more than R100. The total is the sum of the number ordered (numord) times the quoted price on each of the order lines for the order.

3.1 Write the view definition ORDTOT.
3.2 Write an SQL query to retrieve the order number and order total for all orders whose total is over R100.
3.3 Convert the query from 3.2 to the query that will actually be executed.

SYSTEM CATALOG

Information about the database is maintained in the system catalogs. The system catalogs are as follows:

systables	describes database tabels
syscolumns	describes columns in tables.
sysindexes	decsribes indexes on columns.
systabauth	identifies table-level privileges
syscolauth	identifies column-level privileges
sysdepend	describes how views depend on tables
syssynonyms	lists synonyms for tables
sysusers	identifies database-level privileges
sysconstraints	records constraints placed on database tables
syssyntable	used for mapping of synonyms

Explore the systables, syscolumns tables in the system catalog.

SELECT * FROM SYSTABLES, and do the same for syscolumns.

WRITE SQL STATEMENTS TO ANSWER THE FOLLOWING QUERIES

1. List the name and owner of all tables known to the system.

2. List all the columns in the customer table as well as their associated datatypes

3. List all tables that contain a column called SLSRNUMB.

Memorandum IS III 8 March 1991

Question 1

```
Select CAR where MANUFACTURER = "M1" AND COLOUR = "RED" AND
YEAR = 1980
Project Result by CARID and MODEL
```

X(CAR.CARID, CAR.MODEL) : CAR(CAR.COLOUR = "RED" ^ CAR.MANUFACTURER = "M1" ^ CAR.YEAR = 1980)

```
Select HOME where PLOT_SIZE = 1000m^2 giving Result
Join Result, Person_Home over Hometype giving Result1

Select CAR where Manufacturer = "M2" giving Result2
Join Result2, Person_Car over Carid giving Result3

Join Result3, Person over Idno giving Result4
Select Result4 where Age > 40 giving Result5
Project Result5 by Idno

Join Result5, Result1 over Idno giving Result6
Project Result6 by Home_Type giving Result7
```

X(Home.Hometype) : Person_home(Person_Home.Idno = Person.Idno and Person_Home.Hometype = Home.Hometype and Home.Plot_Size=1000m^2 and Person.Age > 40)
Person_Car(Person.Idno = Person_Car.Idno and Person_Car.Carid = Car.Carid and Car.Manufacturer = "M2")

```
Select Home where No_of_rooms = 4 and Swim_pool = "N" giving
Result1
Join Result1, Person_Home over Hometype giving Result2
Join Result2, Person over Idno giving Result3
Select Result3 where Marital_Status = "Unmarried" giving
Result4
Project Result4 by Name, Hometype
```

X(Person.Name, Home.Hometype) : Person_home(Person_Home.idno = Person.Idno and Person_home.Hometype = Home.Hometype and Home.No_Of__rooms = 4 and Home.Swim-pool = "N" and Person.Marital_status = "Unmarried")

Question 2

2.1 Every dog has bitten a postman
2.2 Every dog has bitten every postman
2.3 Everyone has a parent.

VRAAG 4

4.1.1 Repeating group is a group of attributes that will carry values for a specific attribute, e.g. candidate key, within the relation

4.1.2
A1	ART1	50	6
A1	ART2	70	8
A1	ART3	80	4

4.1.3 IT IS IN 0NF OR IN NO NORMAL FORM.

4.1.4 **R1 (ALBUM NO**, COST, ON_HAND)

R2 (**ALBUM NO,ARTIST NO**)

4.2.1 FIRST NORMAL FORM

4.2.2 FUNCTIONAL DEPENDENCY DOES EXIST. EVERY ATTRIBUTE IS DEPENDENT ON THE KEY, BUT THE PROJECT DETERMINES THE SUPERVISOR AND THUS NOT IN 2NF.

4.2.3 INSERT
CANNOT INSERT A SKILL WITHOUT HAVING AN EMPLOYEE OR NO PROJECT WITHOUT HAVING AN EMPLOYEE

DELETE
UPON DELETING AN EMPLOYEE THE SKILL AND PROJECTS MAY ALSO DISAPPEAR

UPDATE
THE POSITION NAME WILL RE-APPEAR FOR EACH EMPLOYEE LINKED TO IT.

4.2.4

SECOND NORMAL FORMS

R1 **(EMP# ,** POS, AGE)

R2 (EMPNO, SKILL, YRS)

R3 (**EMPNO, SKILL, PROJ,** SUPR)

STILL TRANSITIVE DEPENDENCY BETWEEN **PROJ** AND **SUPR**

THIRD NORMAL FORM

R3.1 (**EMPNO, PROJ**)

R3.2 (**PROJ,** SUPR)